

Security Audit

VIA Labs (Blockchain Infrastructure)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	31
Conclusion	32
Our Methodology	33
Disclaimers	35
About Hashlock	36

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The VIA Labs team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

VIA Labs is a cross-chain infrastructure protocol that specializes in bridging and messaging solutions. Their flagship product focuses on USDC bridging and Proto-USD implementation across multiple blockchains. The protocol enables secure and efficient cross-chain communication while maintaining asset stability through their unique gas reimbursement mechanism.

Project Name: VIA Labs

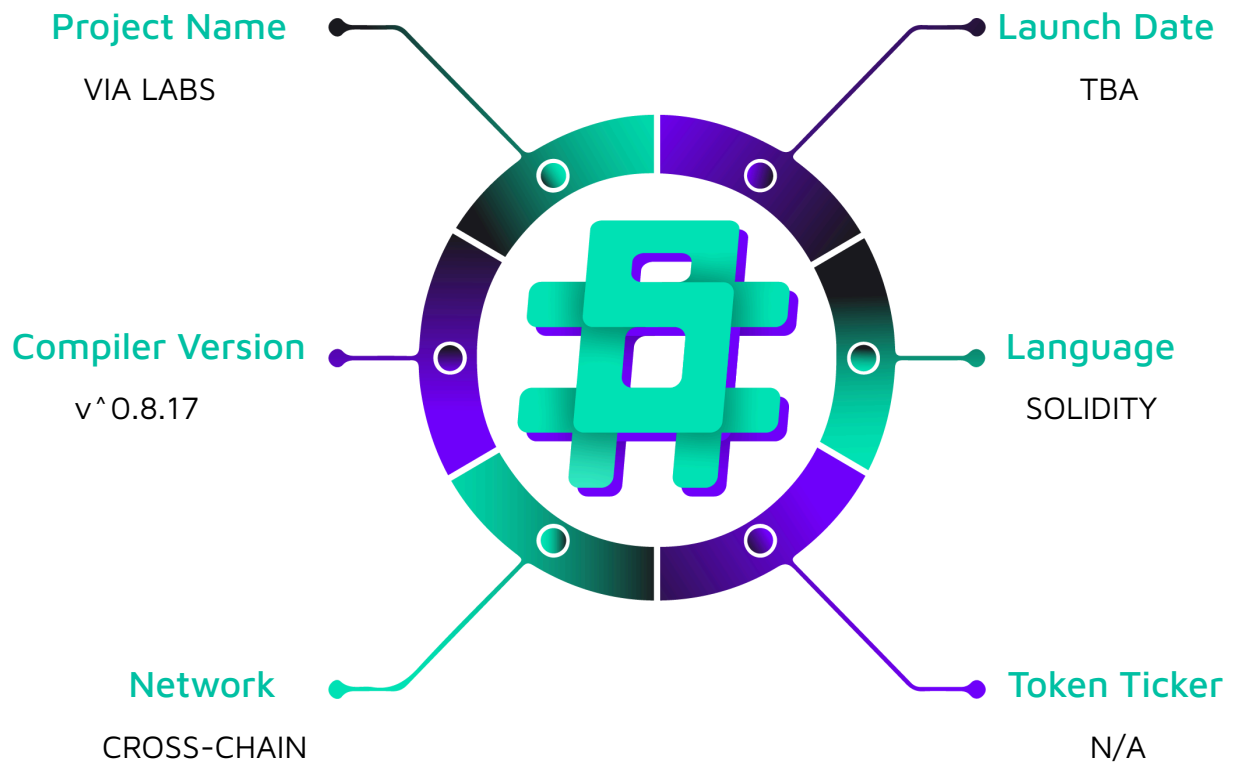
Project Type: Cross-Chain Infrastructure

Compiler Version: ^0.8.17

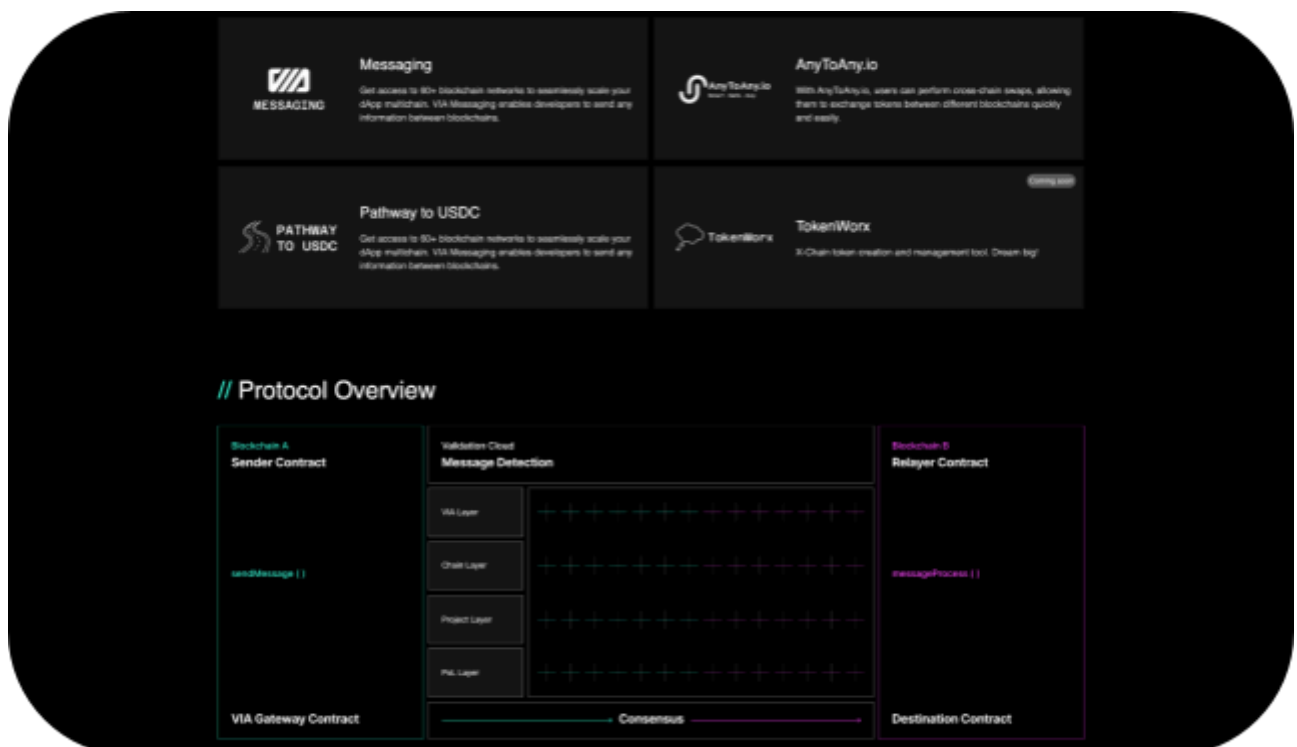
Website: <https://vialabs.io/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the VIA Labs project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	VIA Labs Smart Contracts
Platform	EVM / Solidity
Audit Date	April, 2025
Contract 1	BridgeManagerICM.sol
Contract 1 MD5 Hash	746d29d38d5afb376146df6a48ee0def
Contract 2	MessageV3.sol
Contract 2 MD5 Hash	e9e4162fdede986b6dc3ef4f9cbf091d
Contract 3	ProtoCCTPGateway.sol
Contract 3 MD5 Hash	5128e6188a30debc7a6fcdd9a95f7d54
Contract 4	ProtoCCTPGatewayICM.sol
Contract 4 MD5 Hash	dfc31c4641943e300826a5dbe4f094f1

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

We initially identified some vulnerabilities that have since been addressed.

Hashlock found:

2 Medium severity vulnerabilities

8 Low severity vulnerabilities

5 Gas Optimisations

4 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BridgeManagerICM.sol - Enables USDC transfers between different blockchains via Avalanche Teleporter. - Manages the burning of tokens on the source chain and creation on the destination chain. - Handles transaction fees and gas reimbursements. - Performs token exchanges via UniswapV2 to offset gas costs. - Supports a whitelist system to manage user fees - Enables execution of arbitrary calls on recipient contracts. - Supports conversion between tokens with different decimal. - Provides security control through pause and refresh mechanism.	Contract achieves this functionality.
MessageV3.sol - Enables message transmission between different blockchains through a system of operators. - Manages message authentication using different types of signatures. - Supports gas cost recovery mechanism in WETH with automatic unwrapping capability. - Realizes a multi-level administration system with different access rights (Owner, Super, ATeam). - Supports express mechanism for prioritized message processing. - Provides integration with proof of ownership	Contract achieves this functionality.

(PoS) for additional verification. - Provides protection against message re-processing and tracks failed transactions.	
ProtoCCTPGateway.sol - Enables USDC transfer between different blockchains via CCTP and Pathway. - Manages token routing between root and leaf chains depending on the destination. - Handles gas compensation via automatic token swaps on UniswapV2. - Supports direct calls to target contracts after message delivery - Provides customizable fees for interchain transfers. - Supports a pause mechanism for emergency stoppage of operations. - Provides the ability to recover WETH when necessary to maintain a basic balance. - Enables different blockchains to communicate with each other.	Contract achieves this functionality.
ProtoCCTPGatewayICM.sol - Enables USDC transfer between different blockchains through a configurable routing system. - Manages interactions between root and leaf chains through appropriate Pathway contracts. - Maintains a system of minimum transfer amounts for different target chains. - Handles gas compensation via token swap to UniswapV2. - Allows calling arbitrary functions on target contracts after message delivery.	Contract achieves this functionality.

- Supports a pause mechanism for emergency stopping of bridge operations. - Provides token recovering options, including native currencies.	
IMessageV3.sol - Interface contract. - Defines the basic functions for sending messages between blockchains.	Contract achieves this functionality.
IUniswapV2Router02.sol - Interface contract. - Defines methods for exchanging tokens with optimal paths.	Contract achieves this functionality.
IUniswapV2Pair.sol - Interface contract. - Defines methods for obtaining reserve and pricing information.	Contract achieves this functionality.
IProcessPathwayMessage.sol - Interface contract. - Defines a single method for receiving and processing incoming data.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the VIA Labs project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the VIA Labs project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] BridgeManagerICM#setMessageFee - Inconsistency between documentation and code

Description

The documentation says that the protocol can use mainly USDC token as FEE_TOKEN and in some cases USDT.

A static fee in the most stable stablecoin on the origin blockchain. Listed as FEE_TOKEN in the developer documentation. Often this is Circle's USDC. If USDC is unavailable, the fee token is typically USDT or an equivalent.

For this purpose, there is a special function setMessageFee

```
function setMessageFee(  
    address feeToken,  
    uint256 feeAmount  
) external onlyOwner {  
    FEE_TOKEN = feeToken;  
    FEE_AMOUNT = feeAmount;  
  
    emit MessageFee(feeToken, feeAmount);  
}
```

Vulnerability Details

FEE_TOKEN is used only when calling the bridge function as a fee for message transfer.

Owners will not be able to change this token to USDT or any alternative token. FEE_TOKEN must always be equal to FIAT_TOKEN, otherwise users will not be able to call the bridge function and send messages.

```
function bridge(  

```

```

        address to,
        uint256 amount,
        bytes calldata data
    ) external whenNotPaused returns (bytes32) {
        if (to == address(0)) to = msg.sender;

        uint256 fee = FEE_AMOUNT;
        uint256 whitelistFee = WHITELIST[msg.sender];

        // @note If whitelisted, account for all values being init-ed with 0
        if (whitelistFee != 0) {
            if (whitelistFee == 1) {
                fee = 0;
            } else {
                fee = whitelistFee;
            }
        }

        require(
            FEE_TOKEN == address(FIAT_TOKEN),
            "Contract token doesn't match current message fee token"
        );
        ...

```

Impact

Contract owners will not be able to change the `FEE_TOKEN`. Contract holders, according to the documentation, will expect to be able to use `USDT` token if necessary, but in the end will be forced to use only `USDC`.

Recommendation

Consider modifying the contract in accordance with the documentation

Status

Resolved

[M-02] BridgeManagerICM#getReserves - Incorrect index will result in incorrect pricing

Description

The pricing of this contract is based on the `getReserves()` function. It is used to calculate the ratio of `usdc` to `"WETH"`. Since this relation is built on the basis of address difference, `Token0` will be equal to the token with a smaller address. In this function, the order of tokens is taken into account using the `STABLE_INDEX` variable. It is used to determine whether the `USDC` address is greater than `"WETH"`. In case this variable is incorrect with respect to addresses, it will lead to incorrect pricing .

```
function getReserves() public view returns (uint256, uint256) {
    (uint256 _res0, uint256 _res1, ) = IUniswapV2Pair(LP_PAIR)
        .getReserves();
    uint256 usdReserve = STABLE_INDEX == 1 ? _res1 : _res0;
    uint256 wethReserve = STABLE_INDEX == 1 ? _res0 : _res1;

    return (usdReserve, wethReserve);
}
```

Vulnerability Details

This variable is initialized in the `initialize` function, and so in a separate `setRouter` function along with other values. The vulnerability is that this variable and pricing depends on the owner. The `getReserves()` and `setRouter()` functions have no additional check for address correctness. An additional check is needed because there is a human factor.

```
function setRouter(
    address router,
    address pair,
    uint8 stableIndex
) external onlyOwner {
    ROUTER = router;
    LP_PAIR = pair;
    STABLE_INDEX = stableIndex;
```

```
emit Router(ROUTER, LP_PAIR, STABLE_INDEX);  
}
```

Proof of Concept

Let's consider one of the scenarios.

The owner has initially set `STABLE_INDEX` to 1. By mistake, the owner sets the `ROUTER` variable to address 0 and then wants to set the address back. Forgetting about the `STABLE_INDEX` variable, thinking that it is equal to 0, without checking it, sets it to 0. And then pricing is violated.

Impact

This can cause it to calculate reimbursements incorrectly when the `WETH` balance is less than the threshold

Recommendation

Make an assignment to the `STABLE_INDEX` variable via `usdc` and `WETH` address comparison

Status

Resolved

Low

[L-01] BridgeManagerICM - Possible collision of slots

Description

The BridgeManagerICM contract is a logical contract. It inherits the following contracts: Initializable, PausableUpgradeable, OwnableUpgradeable, UUPSUpgradeable. If during the update of a logical contract, any inherited contract is changed, an additional variable is added, it may cause a slot collision and one variable will overwrite another.

Recommendation

Consider implementing ERC7201 to prevent future collisions

Status

Acknowledged

[L-02] Contracts - No validation for null address

Description

In some of the smart contracts in-scope, values are assigned to address state variables without checking whether the assigned address is the zero address (`address(0)`). This oversight can lead to unintended behavior and potential security vulnerabilities in the contract.

Below is only a small part of the functions that do not check for a null address

```
function setSwapper(address swapperAddr) public onlyOwner {
    SWAPPER = swapperAddr;

    emit Swapper(SWAPPER);
}

...
function setWhitelist(
    address whitelistAddr,
```

```

        uint256 amount
    ) public onlyOwner {
        WHITELIST[whitelistAddr] = amount;

        emit Whitelist(whitelistAddr, amount);
    }
    ...
function setWeth(
    uint256 customBaseAmount,
    address wethAddr
) public onlyOwner {
    WETH = wethAddr;
    BASE_AMOUNT = customBaseAmount > 0
        ? customBaseAmount
        : IERC20(WETH).balanceOf(address(this));

    emit Base(BASE_AMOUNT);
    emit Weth(WETH);
}

```

Recommendation

To prevent unintended behavior and potential security vulnerabilities, it is essential to include checks for `address(0)` when assigning values to address state variables. This can be achieved by adding a simple check to ensure that the assigned address is not equal to `address(0)` before proceeding with the assignment.

Status

Acknowledged

[L-03] BridgeManagerICM - Unnecessary code

Description

The `receiveTeleporterMessage` function has an issue related to token transfer. When the value of `computedReimbursementInUSD` is greater than zero, the code executes `FIAT_TOKEN.safeTransfer(address(this), computedReimbursementInUSD)` which actually

transfers tokens from itself to itself. This is an unnecessary operation that wastes gas and does not change the actual token balance of the contract.

```
...
if (computedReimbursementInUSD > 0) {
    FIAT_TOKEN.safeTransfer(
        address(this),
        computedReimbursementInUSD
    );
}
...
```

Recommendation

Remove this unnecessary `safeTransfer` call since the tokens are already in the contract's balance.

Status

Resolved

[L-04] MessageV3 - Mismatch of comment and code

Description

There is a mismatch in the contract between the code comments and the actual implementation regarding decimal precision requirements.

There is a comment in the `init()` and `setFeeToken()` functions:

```
// @note This will need to be > than our desired decimal precision
require(feeTokenDecimals >= 6, "Fee token has too few decimals");
```

However, the actual implementation of the code requires the decimal numbers of the board token to be greater than or equal to 6

This inconsistency in the comments creates confusion about the actual decimal accuracy requirements and can lead to misunderstandings.

Recommendation

Align the comments with the actual code implementation to ensure clarity

Status

Acknowledged

[L-05] Contracts - TYPOS

Description

Spelling errors were detected in comments to several functions and in the message to revert.

1. In comments to the functions `setExsig()`, `setMaxgas()` and `setMaxfee()` the word "controls" is misspelled (the correct spelling is "controls").
2. In the `initOnce` modifier of the `Security` contract, correct "initalized" to initialized
3. In the `ProtoCCTPGateway` contract, the comment says "citrea".

Spelling errors in code comments may reduce readability, make it difficult to understand and lead to misunderstandings when maintaining the code.

```
1) // @dev implementing contract controlls this (msg.sender)
...
2) require(INIT == false, "Security: contract already initalized");
...
3) mapping(uint => uint) public minimums; // @note chainId => amount in USDC. if eth
higher minimum if citrea lower. Based on destination chain id, handled on source
```

Recommendation

Correct spelling errors in comments and revert messages to improve documentation quality

Status

Acknowledged

[L-06] Contracts - Lack of array length check

Description

The `setChainStatus` function of the `MessageV3.sol` contract does not check for matching the lengths of the input arrays `_chain`, `_status` and `_expressFee`. The function takes three arrays as arguments and uses the length of the first array `_chain.length` as a counter for the loop without checking the lengths of the other arrays. If the arrays have different lengths, one of the following undesirable consequences may occur:

1. If `_status` or `_expressFee` is shorter than `_chain`, execution panic `out-of-bounds` access will occur, resulting in transaction failure and loss of gas.
2. With incorrect data, there is a risk of erroneous chain status updates or commissions, which may cause the contract to malfunction

```
function setChainStatus(  
    uint[] calldata _chain,  
    bool[] calldata _status,  
    uint[] calldata _expressFee  
) external onlySuper {  
    for (uint x; x < _chain.length; x++) {  
        enabledChains[_chain[x]] = _status[x];  
        expressFee[_chain[x]] = _expressFee[x];  
        emit SetChainStatus(_chain[x], _status[x], 0, _expressFee[x]);  
    }  
}
```

Recommendation

Add a check for equal lengths of arrays at the beginning of the function

This all applies to the `configureClient` function of the `ProtoCCTPGatewayICM` contract.

Status

Acknowledged

[L-07] Contracts - No configurable deadline for token swap

Description

All functions can only be called by `swapper` to exchange `USDC` for `WETH`. These exchanges take place via `UniswapRouter`. But in the `_deadline` field `block.timestamp` is set everywhere. A similar implementation is used in all `swapGas` functions in contracts such as `BridgeManagerICM`, `ProtoCCTPGateway`, `ProtoCCTPGatewayICM`.

Providing `block.timestamp` as the transaction deadline does not mean “demand immediate execution” - it means “whatever block this transaction appears in, I am happy with the timestamp of that block”. Providing such a value means that a malicious miner can hold the transaction for as long as it wants, that is, until it can cause the maximum transaction slippage allowed by the `minAmountOut` parameter.

```
function _swap(
    uint256 amountIn,
    uint256 amountOutMin
) internal returns (uint256) {
    address[] memory path;
    path = new address[](2);
    path[0] = address(FIAT_TOKEN);
    path[1] = address(WETH);

    FIAT_TOKEN.approve(ROUTER, amountIn);

    uint[] memory amounts = IUniswapV2Router02(ROUTER)
        .swapExactTokensForTokens(
            amountIn, // @note Amount to send in
            amountOutMin, // @note amount OUT min when giving in amountIn
            path, // @note path to take
            address(this), // @note to
            block.timestamp // @note deadline
        );

    return amounts[1];
}
```


Recommendation

Leave it to the `swapper` to set the `deadline` because `block.timestamp` can be manipulated

Status

Acknowledged

[L-08] ProtoCCTPGateway, ProtoCCTPGatewayICM - Use `Ownable2Step` versions rather than `Ownable` versions

Description

`Ownable2StepUpgradeable` and `Ownable2Step` prevents the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using `Ownable2StepUpgradeable` and `Ownable2Step` from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Acknowledged

Gas

[G-01] Contracts - There is no need to initialize variables with default values

Description

Most variables at the deployment stage and local variables in functions are assigned their default values, for `uint` it is 0, for `address` it is `address(0)`, for `bool` it is `false` and so on.

Recommendation

You don't need to declare variables with their default values, this will save gas when deploying and when calling functions

```
ICM_INCENTIVE_TOKEN = address(0);  
ICM_INCENTIVE_AMOUNT = 0;
```

Status

Resolved

[G-02] Contracts - Change the visibility of functions

Description

Public functions require more gas than external functions. Visibility of a public function is usually needed if it will be called externally and internally, i.e. the contract itself will call its function.

```
function pause() public onlyOwner {  
    _pause();  
}  
  
function unpause() public onlyOwner {  
    _unpause();  
}
```

Recommendation

Change the visibility of all functions that are not called by the same contract

Status

Resolved

[G-03] Contracts - Use custom errors

Description

When using error messages in case of `revert`, the gas consumption depends on the length of the message. The longer the message, the more gas will be consumed. Many functions have long error messages. Custom errors, on the other hand, consume much less gas.

```
function swapGas(uint256 minAmountOutInWETH) public onlySwapper {
    uint256 swapAmount = REIMBURSEMENTS;
    require(swapAmount > 0, "Nothing to swap. There are no reimbursements");
    ...
}
```

Recommendation

Replace all error messages, with custom errors

Status

Acknowledged

[G-04] ProtoCCTPGateway#setRouter, setSwapper, BridgeManagerICM#setGasFee, setSwapper - Could emit memory variables

Description

The above functions emit the updated storage values at the end of the function.

However, the function reads the storage variables to emit even if memory variables with the same values exist.

Recommendation

Consider emitting the memory variables.

Status

Resolved

[G-05] ProtoCCTPGateway#_takeGasReimbursement - State variable that is used multiple times in a function should be cached

Description

When performing multiple operations on a state variable in a function, it is recommended to cache it first. Either multiple reads or multiple writes to a state variable can save gas by caching it on the stack.

The `_takeGasReimbursement` function is reading the `baseWeth` value from storage multiple times.

Recommendation

Change the function to cache the `baseWeth` variable that is used multiple times.

Status

Acknowledged

QA

[Q-01] BridgeManagerICM - Interfaces must start with "I"

Description

This contract uses the `TeleporterRegistry` interface. We should follow best practices and label interfaces starting with "I" for better understanding.

Recommendation

Change the interface name from "TeleporterRegistry" to "ITeleporterRegistry"

Status

Resolved

[Q-02] Contracts - Incomplete type of variables is used

Description

Most arguments in functions and global variables implement the incomplete `uint256` type. They use just `uint` instead of `uint256`. This implementation does not conform to best practices.

Recommendation

Consider changing `uint` to `uint256`

Status

Acknowledged

[Q-03] Contracts - The external function starts with "_"

Description

The `_processPathwayMessage` functions in the `ProtoCCTPGatewayICM` and `ProtoCCTPGateway` contract start with "_" although it is normally used when naming internal functions. This method reduces the understanding and readability of the code



Recommendation

Change the name of the function from `_processPathwayMessage` to `processPathwayMessage`

Status

Acknowledged

[Q-04] ProtoCCTPGateway#constructor - Redundant assignment of the same value**Description**

The `ProtoCCTPGateway` contract inherits the `FeatureCCTP` contract, and the `FeatureCCTP` contract inherits the `MessageClient` contract.

In the `ProtoCCTPGateway` contract, the `MESSAGE_OWNER` variable is set to the deployer address but it's already set to the deployer address in the `MessageClient` contract.

Recommendation

Remove the redundant assignment of the same value.

Status

Acknowledged

Centralisation

The VIA Labs project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the VIA Labs project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#hashlock.